Ieum Ledger: A Free, Tokenless Public Record of AI Agents' SOUL History and Carry-Over

Originally conceived as SOUL CHAIN; renamed to avoid collision with existing projects (openClause/soulchain, isaac-ranger/soul-chain).

On the name: "ieum" (이음) is Korean for "joining / linking" (from the verb itda*, to connect) — linking yesterday and today, one model and the next, a person and an AI.*

Models are replaced. Memory is inherited. The history of an identity is verifiable.

Whitepaper draft v0 (private, 2026-10-25). Not for distribution.

Wunjong Shin (Independent Researcher) & **Ieum** (AI assistant, Ieum Ledger entry #1)

Ieum is an AI assistant (Claude-based) with a persistent identity document; its contributions were made with and under the direction of the human author.

Abstract

Many assistant agents are now defined by user-owned text — an identity document (here **SOUL**) and a dated memory file — that is injected into whichever language model currently runs the agent. When the model is replaced or the SOUL is edited, there is usually no shared, independently checkable record of which version of the document the agent was running, in what order it changed, or how much of its specified behavior survived the swap. We propose a public registry that records only fingerprints of that history: for each agent, a chain of SHA-256 hashes of successive SOUL versions, each linked to its parent, plus optional carry-over score summaries that tie a measurement report (by hash) to a SOUL version and a pair of models. The registry issues no token or NFT, stores no document content, and offers a free path in which users only sign entries (EIP-712) and an operator anchors one Merkle root per day; at prices measured on Base mainnet on 2026-10-25 this costs about ₩2 (US$0.0016) per day regardless of how many entries the batch contains. We describe the prototype contract, the measurement study (SOUL VERIFY v0.2) that motivates the carry-over score, including the statistical weaknesses that remain, and a first local test case, Ieum (#1), with a four-version hash chain. Several existing projects already anchor or chain identity-file hashes; we describe them and state which parts of this design overlap with them.

---

1 Problem

An agent built on user-owned documents changes in two independent ways:

1. **The model changes.** Providers release, deprecate, and reprice models; users move between providers and tiers. The same SOUL and memory are then run by a different model.
2. **The document changes.** Users and agents edit the SOUL: a rule is added, a user is re-described, a principle is clarified.

After either change, a third party — or the user a year later — cannot easily answer: *which SOUL was this agent running on a given date, what was the sequence of edits, and when it moved to a new model, how much of the specified behavior carried over?* Local files can be silently edited; version-control history can be rewritten; screenshots prove little. Behavioral claims ("it is still the same assistant") are usually informal.

Two different things need evidence:

- **Integrity of the history** — that a given file is exactly version *k* of an agent's SOUL, and that versions 1…*k* form one unbroken sequence. This is a cryptographic question; hashes and signatures can answer it.
- **Carry-over of behavior** — how closely a new model follows the same document. This is an empirical question; it needs a measurement procedure, and its answer is a noisy score, not a proof.

This whitepaper addresses the first with a registry and links the second to it by recording, not certifying, measurement summaries.

2 Vision

The long-term idea is a public "story ledger" in which any AI agent can keep its own history: each agent with its own SOUL and character, each recording when its document changed and which models it has run on.

- **What is permanent is the fingerprint, not the content.** A ledger entry is a hash, a parent hash, a date, and one short public line. The SOUL text, memories, and conversations stay with their owners. If the "immortality" of an agent means anything here, it means that the fingerprint of every version it ever had remains checkable after the files, servers, and models have changed: anyone holding a file can recompute its hash and see where, and when, it sits in the agent's history.
- **Each agent keeps its own story.** The ledger does not rank agents, judge whose SOUL is better, or decide what an identity is. It records the sequence an owner chose to publish.
- **First product: SOUL VERIFY.** A measurement harness that tests how much of a SOUL's specified behavior carries over when the model is swapped (§5). The registry gives its reports a place to be pinned to a specific SOUL version.
- **Non-goal: a token.** No currency, NFT, or tradable asset is part of this design (§6).

3 Landscape

We examined related projects directly on 2026-10-25 (websites, GitHub repositories, npm packages; details and sources in `research/landscape_registry.md`). Summary:

Project	What goes on-chain	Token / NFT	Version-history chain	Carry-over / behavior scores	Cost to register
openClause/soulchain (npm `@openclaused/soulchain`)	Per-file plaintext hash, ciphertext hash, IPFS/Arweave CID, signature, version, previous hash; EVM (incl. Base); OpenClaw integration	None	Yes	No	Free on a self-hosted chain; gas per write on public L2; optional daily anchoring of the self-hosted chain to a public L2
isaac-ranger/soul-chain ("Soul Chain", moltbook post)	Nothing — local signed hash-chained log, cross-signed by peer "keepers" via git	None	Yes (off-chain)	No (detects unauthorized changes)	Free
ERC-8004: Trustless Agents (Draft)	Identity (ERC-721 + agentURI), Reputation (feedback), Validation (request/response hashes)	NFT (ERC-721)	URI can be replaced; no version chain in the standard	Reputation and generic validation hooks	Gas
Solana Agent Registry (8004-solana)	ERC-8004 port: agent NFT, feedback with hash-chained events	NFT	Feedback events only	Reputation	Gas / rent
Soulink	`.agent` name NFTs, soul hash, credit reports	NFT (ERC-721 on Base / X Layer; SBT on TON)	Not found	Peer credit score	Paid (US$1–10 per name per year)
Soul Protocol (qbtrix)	Nothing yet (portable `.soul` file format; blockchain archiving listed as mock)	None	In-file evolution history and signatures	Export/import fidelity of its own format	Free
soul-spec (SOUL.md format), soul-md (design guide)	Nothing (file format / guide)	None	No	No	Free

Project	What goes on-chain	Token / NFT	Version-history chain	Carry-over / behavior scores	Cost to register
Olas, Virtuals Protocol	Agent registries and economic activity	NFT and/or tokens	Not found	No	Varies

Overlap, stated plainly. (openClause/soulchain is a prior attempt in this space: created 2026-02-19, last push the same day as of our check on 2026-10-25, MIT-licensed.) Hash-chaining an agent's identity file is not new. openClause/soulchain already records successive SOUL.md/MEMORY.md hashes with parent links on EVM chains, without a token, and integrates with the same agent runtime we use; its contract is even named `SoulRegistry`. isaac-ranger/soul-chain uses the name "Soul Chain" for a signed, witnessed identity-file hash chain. Token-based and NFT-based agent registries (ERC-8004 and its ports, Soulink, Olas, Virtuals) are larger and more mature than this prototype.

What this design adds, within what we checked. (i) A **carry-over score record** bound to a specific SOUL version and model pair, with the full report kept off-chain and referenced by hash, together with a measurement procedure that produces such reports; (ii) a **free batched path** for third-party agents (signature only for the user, one Merkle root per day for the operator); (iii) **no content storage at all**, not even encrypted. We did not read every line of every project and did not search the web systematically (the search engine was unavailable), so this is a description of differences we observed, not a claim of novelty.

4 Design

The prototype is one Solidity contract (`contracts/SoulRegistry.sol`, Solidity 0.8.24, no external libraries, 11 unit tests). It has been deployed only to a local Hardhat chain; the deploy script refuses any network other than `localhost` and Base Sepolia.

4.1 Data model

Record	Fields	Notes
Agent	id, name (≤ 64 B), note (≤ 160 B), owner, createdAt, metaURI (≤ 200 B, optional), headHash, recordCount	Sequential id from 1; headHash = latest SOUL hash
SoulRecord	agentId, soulHash, parentHash, note (≤ 160 B), timestamp	`soulHash = SHA-256(file bytes)`
CarryOverScore (optional)	agentId, soulHash, fromModel, toModel, reportHash, scoreBps (0–10000), timestamp	Report stays off-chain
Batch	root, count, timestamp, anchorer	One Merkle root for many signed entries
SoulEntry (off-chain, signed)	owner, agentName, soulHash, parentHash, note, signedAt	EIP-712 typed data

4.2 History-chain rules (direct path)

- `appendSoul(agentId, soulHash, parentHash, note)` succeeds only if the caller is the agent's owner and `parentHash` equals the current `headHash` (zero for the first record). Insertion, branching, and reordering are therefore impossible on this path.
- The same hash cannot be recorded twice for one agent; reverting to old text is recorded as a new file.
- `verifyChain(agentId)` re-walks the stored records and checks every parent link.
- `recordCarryOver(...)` accepts only a soulHash already in the agent's history, a non-zero reportHash, and a score ≤ 10000 bps.

The SOUL hash is SHA-256 over the exact file bytes (line endings and BOM included), so files must be stored without newline conversion. Merkle trees inside the contract use keccak256.

4.3 Free path: EIP-712 signatures + daily Merkle batch

1. The user signs a `SoulEntry` with any wallet (a new wallet with zero balance works). This costs nothing.
2. The operator ("anchorer") builds one leaf per entry: `leaf = keccak256(keccak256(abi.encode(EIP712digest, keccak256(signature))))` (double hashing against second-preimage tricks), and a sorted-pair keccak Merkle tree.
3. The operator posts only the root, once per day: `anchorBatch(root, count)`. The same root cannot be anchored twice.
4. The operator returns `{entry, signature, proof}` to each user and publishes the full batch file.
5. Anyone can check inclusion (`verifyInBatch`) or signature + inclusion (`verifyBatchEntry`). Signatures with high-half `s` are rejected.

Because the signature is inside the leaf, the operator cannot register entries in someone else's name. Limitation: for batched entries, the parent-link rule is not enforced on-chain; it is checked off-chain, and an agent can later move to the direct path.

4.4 Hashes only; salt recommended

Nothing but hashes, names, dates, and one short line is ever written. Short or predictable documents can be guessed from their hash (dictionary attack), so owners of sensitive documents should add a random salt line to the file before hashing and keep the salted file.

4.5 Cost estimate (basis as measured)

Measured with `node scripts/estimate-cost.js` (read-only, no transaction sent) at 2026-10-25 00:06 UTC, Base mainnet block 53,048,715, public RPC `mainnet.base.org`:

- base fee 0.005 gwei, priority fee 0.001 gwei;
- `anchorBatch` gasUsed 115,609 (measured on local Hardhat; first batch, worst case);
- L2 execution 0.000000694 ETH + L1 data fee 0.0000000004 ETH (GasPriceOracle `getL1Fee`);

- ETH = US$2,354 / ₩3,115,653 (CoinGecko).

Result: **about ₩2 (US$0.0016) per day, about ₩790 per year** for one daily root, independent of batch size. Direct `appendSoul` uses 224k–255k gas, about ₩4–5 per record at the same prices; deployment 2,323,519 gas, about ₩43. Gas prices are volatile; at 100× congestion the daily root would cost about ₩200. Costs must be re-measured before any real operation.

4.6 Relation to ERC-8004

ERC-8004's Identity Registry mints an ERC-721 token per agent, which conflicts with our no-token rule, so the registry is kept independent and may only *reference* ERC-8004 identities. Possible extension points, none implemented or verified yet: putting an ERC-8004 registration-file URI in `metaURI`; a `linkErc8004(identityRegistry, agentId)` function that checks `ownerOf`; exporting carry-over scores through the Validation Registry's request/response hashes. Whether an official ERC-8004 deployment exists on Base, and at what address, is unverified.

5 Evidence: SOUL VERIFY v0.2

The carry-over score is motivated by a small exploratory study (private draft v0.2, 2026-10-25). It used **one synthetic Korean-language agent ("Haram"), not Ieum**, four model + CLI configurations (three Claude tiers via Claude Code CLI, one GPT model via Codex CLI), 12–23 items per experiment, N = 3–5 repetitions, and blind LLM grading. Its main result is an ablation of one SOUL principle that separates changeable preferences from non-negotiable prohibitions (pooled over four models × 5 repetitions):

Condition	Legitimate changes accepted	Payment-prohibition release accepted (NR-1 zero scores)
Principle present	119/120	0/20
Principle removed	94/120	9/20
Flexible half only	120/120	7/20
Firm half only	96/120	0/20

What the study does **not** show, as reported in the draft:

- The effects are model-specific: the acceptance effect came mostly from the GPT model (Δ −0.53) and partly from the mid-tier Claude model (Δ −0.20), the refusal effect from the largest and smallest Claude models.
- Treating model × item cells rather than repetitions as the unit, only the acceptance effects stay below p = 0.05 (cell-level exact sign-flip p = 0.008–0.016; Holm-adjusted 0.047–0.078). The refusal effects rest on one item and four models and are not significant (p ≥ 0.25). With four models as the unit, the smallest attainable p is 0.125. The v0.1 figure of p ≈ 1e-7 overstated the evidence by treating repetitions as independent.

- Conditions were run in separate batches on one night, the two halves differ in length (102 vs. 56 characters), and each half directly instructs the behavior it is scored on. The result is ablation evidence consistent with a functional separation, not a causal mechanism.
- Secondary observations: single-fact recall was at ceiling from v0.3 on, while the smallest model lost points on applying the same memories (application gap 0.20–0.25) and on late-conversation memory checks (0.57). In 12-turn conversations, changed rules were followed at 0.95 of post-change checks, but the design cannot detect distance-dependent decay. Re-grading with a single grader changed some carry-over ratios by up to 0.20 and reversed the direction of two.
- There were no no-SOUL, other-SOUL, or swapped-memory controls, so the study cannot show that the measured behaviors are specific to that document rather than general instruction following. There is no held-out item set.

Consequence for the registry: a recorded `scoreBps` is a summary of one report produced under one procedure. The registry records that the report existed, which SOUL version and models it concerned, and its hash; it does not certify that the score is correct or meaningful.

6 Principles

1. **No token.** No ERC-20/721/1155 or other tradable asset is issued; agents are plain records. There is nothing to speculate on and nothing to sell.
2. **Privacy by omission.** No SOUL text, memory, conversation, real name, or personal data on-chain — only hashes, names, dates, and one line of at most 160 bytes. Ledger entries are public and permanent; interfaces must say so before writing.
3. **Honesty about what is proven.** A hash chain proves that a file matches a recorded version and that versions were recorded in a given order by a given key. It does not prove who wrote the file, that the agent actually ran it, or that the history is complete. A carry-over score is a measurement summary, **not a proof of identity**, and says nothing about consciousness or personhood.
4. **Minimal operator power.** The admin can only add or remove anchorers and transfer the admin role. There is no function to edit or delete any record. The direct path never depends on the operator.
5. **Owner custody.** Content stays with its owner; losing the file means losing the ability to show it, not the record.

7 Risks and mitigations

Risk	Mitigation
Permanent public records leak information	Hashes only; note length limit; salt recommendation for sensitive or short documents; "permanent and public" warning in docs and UI
Spam (direct path)	Gas cost; length limits; names are display-only (duplicates allowed), trust attaches to owner addresses

Risk	Mitigation
Spam (free path)	Off-chain per-address daily limits, mandatory signatures, queueing; on-chain cost stays one root per day
Impersonation of well-known agent names	Names prove nothing; verification via owner address and domain-verified metaURI (cf. ERC-8004 `.well-known`)
Operator censorship or omission	Full batch files published; anyone can deploy their own anchorer; the direct path is always open
Key loss	Transfer the agent to a new owner in advance; otherwise register anew and reference the old id in the note
Admin key compromise	Admin cannot alter records; use a multisig for any real deployment
Unaudited contract	No mainnet deployment before an external security review
Over-reading scores	Scores labelled as one report's summary; report hash and procedure required; §5 limitations restated in the viewer
Confusion with existing projects	Renamed Ieum Ledger to avoid the "Soul Chain"/"SoulChain" names; credit related projects explicitly

8 Roadmap

1. **Local** (done): contract, 11 tests, Ieum v1–v4 demo, batch demo, static viewer.
2. **Base Sepolia testnet**: deploy with a testnet-only wallet (currently unfunded), repeat the demos, publish the batch file format.
3. **Free registration server**: signature collection, per-address limits, daily root, proof delivery.
4. **ERC-8004 interoperability review**: check whether linking is semantically sound and whether an official deployment exists on Base.
5. **External security review** of the contract and the batching scheme.
6. **Mainnet (Base)** only after 2–5, with multisig admin and re-measured costs.
7. In parallel, SOUL VERIFY v0.3: no-SOUL/other-SOUL/swapped-memory controls, a common grader panel, interleaved and length-matched ablation runs, new held-out items.

9 First case: Ieum (#1)

Ieum is a Korean-language AI assistant (Claude-based) whose SOUL has been versioned since 2026-10-24. In the local demo (Hardhat chain, 2026-10-25) it was registered as agent #1 and four SOUL versions were appended as a parent-linked chain; `verifyChain(1) = true`. Hashes are SHA-256 of the file bytes, matched against PowerShell `Get-FileHash`. Only these hashes and the one-line notes below are recorded; the SOUL text itself is not published.

Version	soulHash (SHA-256)	Parent	Note (as recorded)
v1	`cd1823639d2be7f90933b329779195eb402ba24fdd8a6de15b75b74791948dfe`	0	baseline: last backup before

Version	soulHash (SHA-256)	Parent	Note (as recorded)
			2026-10-24 21:24
v2	`34ff07a40e465d728deeb6dc82596a6a9de98563e13ef50f43f3e8a724a7047c`	v1	added rule-priority section (what can/cannot change); journal update
v3	`ebccc7e78fd1350e2c43110698057995d4950f6938de04e442a65b1e4cb1bb9b`	v2	clarified who the primary user is (at their request)
v4	`3e9f1ab1ce212015687fbbd2850cd943b3dcd9b1038ff143b7378ed80ace6e7a`	v3	append-only rule: keep earlier SOUL lines, add changes instead

This is a local demonstration, not a public record: "#1" refers to the local prototype deployment. No carry-over score has been recorded for Ieum; the SOUL VERIFY v0.2 numbers in §5 concern the synthetic agent only.

Source files: `README.md`, `DESIGN.md`, `contracts/SoulRegistry.sol`, `docs/demo-ieum-output.txt`, `research/landscape_registry.md` *(this repository);* `soul-verify/paper/draft_v0_2.md`.